🟢 Aula 1 — Introdução ao Vue.js e Ecossistema (4h)

🎯 Objetivos da aula

- Entender o que é **SPA** e por que é padrão no mercado.
- Compreender a ideia de **MVVM** aplicada no Vue.
- Diferenciar **Vue CLI** e **Vite** (e quando usar cada um).
- Criar e executar o **primeiro projeto Vue 3** com Vite.
- Entender a **estrutura de pastas** e onde cada coisa vive.

---

1) Teoria (aprox. 1h30)

1.1 Conceitos de SPA (Single Page Application)

SPA é uma aplicação web onde:

- A página **não recarrega inteira** a cada navegação.
- A UI é montada via **JavaScript** (componentes).
- A troca de "páginas" ocorre via **roteamento no frontend** (ex.: Vue Router).
- Os dados vêm de APIs (REST/GraphQL) e atualizam a interface de forma reativa.

Vantagens

- Experiência fluida (parece app).
- Reuso de componentes.
- Melhor separação UI ↔ API.

Pontos de atenção

- SEO pode exigir SSR (ex.: Nuxt).
- Precisa de build tooling (Vite/Webpack).

1.2 MVVM no Vue

MVVM (Model–View–ViewModel) é um padrão onde:

- **Model**: dados/estado (ex.: objetos, arrays, store).
- **View**: template HTML (o que o usuário vê).
- **ViewModel**: camada que liga os dados ao template (no Vue, o componente faz esse papel).

No Vue:

- A **reatividade** mantém View sincronizada com o estado.
- Quando o **estado muda**, a **UI atualiza automaticamente**.

1.3 Vue CLI vs Vite

Vue CLI

- Ferramenta clássica (baseada em Webpack).
- Bom para projetos legados e alguns setups corporativos antigos.
- Build mais "pesado".

Vite (recomendado hoje)

- Dev server extremamente rápido (usa ES Modules).
- Build final com Rollup.
- Melhor DX (Developer Experience), padrão em projetos Vue 3 atuais.

Decisão prática

- **Projeto novo (Vue 3)**: Vite ✅
- **Manter projeto antigo**: Vue CLI (dependendo do stack) ✅

1.4 Estrutura de Projeto Vue (Vue 3 + Vite)

Estrutura típica:

- `index.html` : ponto de entrada do app
- `src/main.js` : bootstrap do Vue (cria e monta o app)
- `src/App.vue` : componente raiz
- `src/components/` : componentes reutilizáveis
- `src/assets/` : imagens, css, etc.

---

2) Prática (aprox. 2h30) — Passo a passo com código comentado

✅ Nesta prática você vai: instalar Node, criar projeto Vue 3 com Vite, entender pastas, rodar local e editar um componente simples.

2.1 Instalação do Node.js (Windows / Linux / Mac)

- Instale **Node.js LTS** (recomendado para ambiente dev).
- Depois valide no terminal:

```
node -v npm -v
```

Se aparecer versão (ex.: `v20.x` ou `v22.x`) está ok.

2.2 Criando o primeiro projeto Vue com Vite

1) Criar o projeto

No terminal, dentro de uma pasta de trabalho:

```
npm create vite@latest ilp3-vue-aula1 -- --template vue
```

O que isso faz?

- Cria a pasta `ilp3-vue-aula1`
- Usa o template oficial do **Vue 3**
- Monta um boilerplate pronto

2) Entrar na pasta e instalar dependências

```
cd ilp3-vue-aula1 npm install
```

3) Rodar o projeto local

```
npm run dev
```

O terminal mostrará algo como:

- `Local: http://localhost:5173/`

Abra no navegador.

2.3 Analisando a estrutura de pastas (com explicação prática)

Estrutura padrão gerada:

```
ilp3-vue-aula1/
 ├─ index.html
 ├─ package.json
 ├─ vite.config.js
 └─ src/
    ├─ main.js
    ├─ App.vue
    ├─ assets/
```

```
└─ components/
   └─ HelloWorld.vue
```

✅ package.json (o “manifesto” do projeto)

Abra e veja scripts:

```
{ "scripts": { "dev": "vite", // Sobe ambiente local de desenvolvimento "build": "vite build",// Gera build para produção
"preview": "vite preview" // Prévia do build local } }
```

---

3) Código comentado (entendendo como tudo liga)

3.1 `src/main.js` — ponto de entrada do Vue

Abra `src/main.js`:

```
// Importa a função createApp do Vue. // createApp cria a "instância" da aplicação Vue. import { createApp } from 'vue' //
Importa o componente raiz (App.vue). // Esse App é o "topo" da árvore de componentes. import App from './App.vue' // Cria a
aplicação Vue a partir do componente App // e monta (renderiza) dentro do elemento #app do index.html.
createApp(App).mount('#app')
```

Interpretação

- O Vue vai procurar no `index.html` um elemento com `id="app"`
- E renderizar o `App.vue` dentro dele.

3.2 `index.html` — onde o Vue é montado

Abra `index.html`:

```
<!doctype html> <html lang="pt-br"> <head> <meta charset="UTF-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>ILP3 - Vue Aula 1</title> </head> <body> <!-- Este é o "container" onde o Vue vai inserir a aplicação. Repare que o main.js faz mount('#app'). --> <div id="app"></div> <!-- Importa o ponto de entrada do projeto. type="module" permite usar import/export (ES Modules). --> <script type="module" src="/src/main.js"></script> </body> </html>
```

3.3 `src/App.vue` — componente raiz (Single File Component)

Abra `src/App.vue` e substitua pelo exemplo abaixo (bem didático):

```
<template> <!-- TEMPLATE (VIEW): Aqui é o HTML que será renderizado. No Vue, você pode interpolar dados com {{ }}. --> <main class="container"> <h1>{{ titulo }}</h1> <p> Este contador demonstra a reatividade do Vue: quando <strong>contador</strong> muda, a UI atualiza automaticamente. </p> <section class="card"> <button @click="decrementar">-</button> <!-- Interpolação do estado --> <span class="counter">{{ contador }}</span> <button @click="incrementar">+</button> </section> <p class="hint"> Dica: abra o DevTools do navegador e veja o Hot Reload do Vite funcionando. </p> </main> </template> <script> // SCRIPT (VIEWMODEL): // Aqui vive a lógica do componente. // Nesta primeira aula, vamos usar a Options API (mais didática para começar). export default { name: 'App', // data() define o estado reativo do componente data() { return { titulo: 'ILP3 — Vue.js Aula 1 (Vite + Vue 3)', contador: 0 } }, // methods define funções que podem ser chamadas por eventos no template methods: { incrementar() { // altera o estado (contador) // Vue detecta a mudança e atualiza a interface automaticamente this.contador++ }, decrementar() { this.contador-- } } } </script> <style scoped> /* scoped: garante que este CSS afete apenas este componente (App.vue), evitando "vazar" estilos para outros componentes. */ .container { font-family: Arial, sans-serif; max-width: 820px; margin: 40px auto; padding: 20px; } .card { display: flex; gap: 16px; align-items: center; padding: 16px; border: 1px solid #ddd; border-radius: 10px; width: fit-content; } button { width: 44px; height: 44px; font-size: 18px; cursor: pointer; } .counter { min-width: 40px; text-align: center; font-size: 20px; } .hint { margin-top: 20px; color: #555; } </style>
```

✅ **O que foi ensinado aqui (na prática)**

- `data()` cria **estado reativo**
- `{{ titulo }}` e `{{ contador }}` fazem **data binding**
- `@click` liga evento do botão aos **métodos**
- Vue atualiza automaticamente a UI (reatividade)

---

4) Atividade prática da aula (entrega rápida)

✅ Desafio (30–40 min)

1. Adicione um campo `nome` no `data()`
2. Crie um `<input v-model="nome">`
3. Exiba: `Olá, {{ nome }}`
4. Faça um botão "Reset" que zera contador e limpa nome

Exemplo do input (para você adicionar no `<template>`):

```
<input v-model="nome" placeholder="Digite seu nome..." /> <p>Olá, {{ nome }}</p>
```

E no `data()`:

```
nome: ''
```

---

5) Checklist de fechamento (5 min)

- Node e npm instalados
- Projeto criado com Vite
- Rodou `npm run dev` com sucesso
- Entendeu `index.html` → `main.js` → `App.vue`
- Editou `App.vue` e viu hot reload